LDPasswd: A Local Differentially Private Solution for Protecting User Passwords in Red Team Engagements

Luke Demi
Department of Cybersecurity
Rochester Institute of Technology

Rishit Gahlot
Department of Cybersecurity
Rochester Institute of Technology

Massimo Marino
Department of Cybersecurity
Rochester Institute of Technology

James Southcott
Department of Cybersecurity
Rochester Institute of Technology

Tri Tran
Department of Cybersecurity
Rochester Institute of Technology

Cayden Wright
Department of Cybersecurity
Rochester Institute of Technology

***Abstract*—This work introduces LDPasswd, a novel library designed to guarantee ε-Local Differential Privacy for password collection within simulated adversarial cybersecurity engagements. Due to their access to passwords within these environments, hypothetical bad-acting red team operators are able to cause undue stress to environment participants by extrapolating bits of their non-environment passwords. To solve this, LDPasswd provides the addition of noise to both the semantics and the diction of a password via the exponential mechanism. This implementation is done in the C programming language, simplifying integration into existing password harvesting tooling. Finally, an experiment utilizing statistical analysis shows that with high values of ε, no utility is lost to red team operators.**

***Index Terms*—Data Privacy, Cybersecurity, Differential Privacy (DP), Local Differential Privacy (LDP), Password Filters, Red Team, Blue Team**

I. Introduction

Cybersecurity practitioners utilize simulated adversarial engagements to explore attack techniques, detection strategies, and incident response workflows against real-world style adversaries. In scenarios with a defensive "blue team" and offensive "red team", these exercises provide a realistic, yet controlled setting for studying security practices and testing security controls. One component of such practices involves the collections of passwords or password hashes. This action emulates credential harvesting and represents how sensitive information is compromised within enterprise infrastructures. However, this collection can accidentally reveal semantic patterns used by users to form passwords. Although this information is confined to the exercise environment and typically destroyed afterwards, a malevolent participant could extract this information and attempt to breach accounts and/or systems that are not related to the competition. This work proposes a novel library, LDPasswd, which provides ε-LDP level privacy for password semantics, ensuring that a malicious red team operator is not able to cause any undue stress to end users.

II. Background

A. Simulated Adversarial Engagements

Simulated adversary engagements, also known as "Red vs. Blue" engagements [1], have proven to be a critical method both for training cybersecurity engineers [2] [3] and assessing their skills [4]. In typical practice, infrastructure is provisioned using virtualization or containerization technologies. The defensive "blue team" maintains simulated service uptime and remediates malicious incidents. The offensive "red team" simulates adversarial behavior by utilizing many techniques found in the MITRE ATT&CK Framework [5], including credential harvesting. In an attempt to secure systems, blue teams change passwords fairly often, which provides the red team an opportunity to observe semi-natural password usage patterns through credential harvesting. Users are instructed to create secure passwords and to not use real credentials/semantically similar credentials to real credentials, however users adhering to this cannot be guaranteed.

B. Malicious Password Filters

Password filters are cybersecurity mechanisms used to enforce specific complexity requirements and discourage risky user habits such as password reuse and the use of easily guessed strings [6]. Traditionally, these filters evaluate plain text passwords against a set of rules, returning whether or not the password is valid. Both the Windows and Linux operating systems provide this feature, through LSASS Notification Package DLLs [7] and Pluggable Authentication Modules [8] respectively. Due to the nature of these password filters, they must have access to the plain text password before it is hashed and stored, making them an attractive target for attackers. MITRE ATT&CK Techniques T1556.002 (Modify Authentication Process: Password Filter DLL) [9] and T1556.003 (Modify Authentication Process: Pluggable Authentication Modules) [10] target the aforementioned operating systems respectively. These techniques have been proven to be used by

multiple Advanced Persistant Threat (APT) Groups [11] [12], as well as wider malware families [13] [14], demonstrating the necessity for their use by simulated adversaries to best emulate real word threats.

C. Differential Privacy

Differential privacy is a mathematical framework designed to quantify and limit the privacy loss of individuals when their data is included in a dataset [15]. A randomized mechanism $\mathcal{M}$ satisfies ε-differential privacy if the probability of any given output is nearly identical whether or not an individual's record is present in a database. This guarantee is controlled by the privacy parameter ε, where a smaller value indicates stronger privacy and vice versa. Previous privacy methods such as k-anonymity relied on generalization and suppression to hide individuals within groups [16]. However, differential privacy provides a provable guarantee regardless of an adversary's background knowledge or computational power.

D. Exponential Mechanism

The exponential mechanism is a ε-DP satisfying mechanism in differential privacy used for selecting a response from a set of possible outcomes [17]. This is especially useful when those outcomes are non-numeric. It operates by assigning a utility score to each possible output within an output space $\mathcal{R}$ defined by a scoring function, which maps a dataset and a candidate output to a real-valued quality score. The mechanism then selects an output with a probability proportional to

$$\exp\left(\frac{\varepsilon \cdot u(D,r)}{2\Delta u}\right)$$

$u(D,r)$ is the utility score function of output $r \in \mathcal{R}$ for dataset D and Δu is the sensitivity of the score function. This ensures high quality (typically similar) results are much more likely to be chosen while maintaining the required privacy budget.

Algorithm 1 The Exponential Mechanism

procedure EXPONENTIALMECHANISM($D, \mathcal{R}, u, \varepsilon$)
  Input: Dataset D, range $\mathcal{R}$, utility function u, privacy parameter ε
  Output: A selected element $r \in \mathcal{R}$
  $\Delta u \leftarrow$ CalculateSensitivity(u)
  $w_r \leftarrow \exp\left(\frac{\varepsilon \cdot u(D,r)}{2\Delta u}\right)$ for each $r \in \mathcal{R}$ ▷ Calculate weights
  $Z \leftarrow \sum_{r' \in \mathcal{R}} \exp\left(\frac{\varepsilon \cdot u(D,r')}{2\Delta u}\right)$ ▷ Normalization factor
  Select $r \in \mathcal{R}$ with probability $\Pr(r) = \frac{w_r}{Z}$
  return r
end procedure

E. Natural Language Processing

Natural Language Processing (NLP) is a field of artificial intelligence focused on the interactions between computers and human language [18]. This often involves large-scale datasets to train models for tasks such as translation, sentiment analysis, and text generation. This field is especially applicable when tokenizing a string into individual words.

Systematically exploring character combinations to identify words within contiguous strings presents a search problem in a state space of possible segmentations. For example, given the string "lemonjumpslide", a tokenization algorithm aims to extract the strings "lemon", "jump", and "slide". NLP techniques, such as the Viterbi algorithm, can use statistical methods to efficiently return the most natural sequence of tokens for a given string based on probabilistic data.

F. Tokenization

To efficiently parse strings, they are broken down into smaller tokens. Several algorithms achieve this including brute force parsing, greedy algorithms, and the Viterbi algorithm. The greedy approach is the simplest heuristic, continually choosing local optimal choices. Brute force parsing evaluates every combination of a string to find dictionary or grammatical matches, but has exponential space complexity [19] [20]. To mitigate the inefficiency of brute force, the Viterbi algorithm finds the most likely sequence of tokens through recursion and backtracking [21]. Hybrid approaches of tokenization allow for the benefit of multiple techniques, depending on the implementation.

III. Related Works

In the past, various mechanisms address privacy in regards to passwords have been proposed. Blocki et al. introduces a method to release password frequency lists without revealing specific passwords of individual users [22]. The authors implement the exponential mechanism for differential privacy. This process perturbs the actual frequencies, ensuring the released data is close to the truth, but mathematically protects each record. This method optimizes minimal distortion of the data and prevents an adversary from using background knowledge (such as from data breaches) to re-identify a user in the list. In offensive cybersecurity exercises, participants utilize these perturbed lists to simulate realistic brute force or dictionary attacks without exposing real credentials.

Panchal et al. improves a previous protocol named Honey Encryption. This generates fake plaintext (honey messages) when an adversary attempts to use an incorrect key [23]. The authors ensure these decoys are contextually plausible, making it difficult for adversaries to determine whether they successfully cracked the password. The work of Panchal et al. employs natural language processing (NLP) techniques such as keyword extraction and transformers. Through such techniques, the decoy messages have no linguistic difference from the real message.

Li et al. quantify how much personal identifiable information (PII) users put into their passwords, which significantly lowers security. The authors highlight traditional password detectors are often ineffective at enumerating semantic-rich patterns. Li et al. introduces a new metric named Coverage, which measures the correlation between a password and PII (such as a name or birthday) [24]. Their work also proposes

the personal-PCFG method to crack passwords by generating personalized guesses.

Simulated cybersecurity exercises lack demonstration of realistic adversaries and simply assume idealized, worst-case scenarios. Swanberg et al. provides a framework to extend differential privacy guarantees to complex attacks goals [25]. These include reconstruction of tabular data or extraction of sensitive information from large language models (LLMs). The framework includes the following metrics:

- The number of records successfully attacked in a large dataset
- How many bits of information a mechanism leaks per person
- An adversary's probability of guessing the data before seeing the private output

The framework also quantifies the amount of risk from an arbitrary number of adversaries simultaneously and models approximate reconstruction of records. This is especially crucial for high-dimensional data such as images or complex text where exact matches are unlikely. In cybersecurity exercises, organizers can quantify the privacy leakage risk of sharing specific types of data (such as passwords). Participants can also use these metrics to determine which datasets are safe to use in the simulation.

Naor et al. identifies popular passwords that should be avoided. The authors ensure security in cases where devices are compromised or servers are malicious. This ensures no single password can be used to attack a large population of users. Security is achieved by combining secure two-party computation with differential privacy. The protocol remains private private even under "continual observation" as the list of popular passwords changes over time [26].

IV. Problem Statement & Research Questions

A. Problem Statement

As seen in present literature, multiple issues arise with red team operators having access to the true plain text values of user passwords (e.g. users including PII in their passwords and using similar passwords for both their real accounts and within the competition). However, this problem can be extended even further. Within engagements, it is not uncommon for end users to learn that their password has been compromised. In an enterprise-style simulated adversary environment, this might be for the purposes of confirming the password was able to be harvested. In an educational-style simulated adversary environment, such as a red vs. blue style educational competition, the end user might learn about their compromised password through various methods meant to make the competition more fun and engaging. Examples of this include the red team directly telling the user about it for fun and users seeing their password in a password wordcloud at an end-of-competition debrief [27]. While these elements of simulated adversary environments better the experience of the engagement as a whole, they could cause undue stress to the end user. Additionally, bad-acting red team operators could use harvested credentials to log into actual enterprise accounts in extreme situations.

B. Research Questions

This study aims to solve the aforementioned problems and answer the following research questions:

- To what extent can a novel ε-Local Differential Privacy (ε-LDP) mechanism be integrated into standard Red Team operations?
- Can the mechanism maintain operational utility without compromising individual privacy guarantees?

V. Methodology

LDPasswd is a library written in the C programming language that provides a perturbation function to achieve ε-LDP protection when collecting passwords in simulated cybersecurity adversary engagements. The code is available on GitHub [28]. There is one function exposed by the library *perturb_password*, which will take a password and a global privacy budget ε as parameters and will perturb the password in-place per this implementation, detailed in Algorithm 2.

Algorithm 2 Password Perturbation Function

procedure PERTURBPASSWORD(P, ε)
  Input: Password string P, privacy parameter ε
  Output: Perturbed password P'
  $\mathcal{T} \leftarrow$ TOKENIZEPASSWORD(P)
  $B_{sem} \leftarrow \frac{\varepsilon}{|\mathcal{T}|+1}$ ▷ Calculate semantic budget
  $B_{dict} \leftarrow B_{sem}$ ▷ Diction budget per token
  for each $\tau \in \mathcal{T}$ **do**
    PERTURBDICTION(τ, B_{dict})
  end for
  $P' \leftarrow$ PERTURBSEMANTICS($\mathcal{T}, B_{sem}$)
  return P'
end procedure

In this algorithm, passwords are first "tokenized" into a list of words, numbers, and special characters. Password perturbation is then split into two distinct categories, semantic perturbation and diction perturbation. Semantic perturbation changes the order of the tokens (e.g. swapping **Password123!** to **123Password!**). Diction perturbation changes the actual value of the tokens, and how this perturbation occurs depends on the token type (whether it is a word, number, or special character). The global privacy budget, ε, is split between these two types of perturbation, seen in Algorithm 2. A privacy budget for semantic perturbation, B_{sem}, as well as a diction budget per token, B_{dict}, will be calculated from ε such that globally, the whole function will satisfy ε-LDP.

In the event that there is only a single token, B_{dict} will simply equal the global privacy budget ε and B_{sem} will equal 0 as no semantic perturbation can occur with one token. Due to modern password complexity requirements, however, this is unlikely to occur.

A. Tokenization

To convert a retrieved password into a list of tokens for parsing, wordlists from the popular password evaluation tool zxcvbn [29] were utilized. These wordlists contain the most frequent English words from Wikipedia entries, names, and common television and film terms, as well as most of their frequencies in their respective datasets. The words are then converted to a trie data structure, which stores sequences of nodes where children nodes contain letters that can form a valid word [30]. The trie data structure is also known as a prefix tree. Each node contains:

- The value of the stored letter
- A boolean indicating whether or not the node represents a valid word
- The cost of the node, which is calculated per

$$\text{Cost}(w) = \log_2(F_t) - \log_2(F_w)$$

  where F_t is the total frequency and F_w is the frequency of the word
- A pointer to the next node
- References to its child nodes

The trie structure allows for efficient retrieval of keys as opposed to a brute-force effort to evaluate all possible letters forming valid words. Additionally, when building the trie (with a script provided in the repository), the *IMPORTANT_WORDS* variable allows for certain words to be emphasized to enhance tokenization for a given engagement environment. For example, when competing in Rochester Institute of Technology's annual beginner red vs. blue competition IRSeC (Incident Response Security Competition) [31], the word "irsec" is often used in passwords, however as it isn't an actual word, its calculated cost would be high. When added to the *IMPORTANT_WORDS* variable, its cost is drastically reduced to improve tokenization for the specific IRSeC environment.

In addition to the wordlists, zxcvbn also provides a static mapping of characters to convert password string into "leetspeak" format. Leet, often stylized as "l337", is a specialized system of modified orthography where letters are replaced with other characters or numbers, preserving the shapes of letters in strings [32]. For example, the string "Password" can be substituted as "P@$$w0rd" by replacing the character 'a' with '@', 's' with '$', and 'o' with '0'. Users often modify their passwords in this manner to meet complexity requirements.

With the trie wordlist structure and leet map, the tokenization algorithm is defined in Algorithm 3. This is a hybrid approach between a Greedy search with a lookahead and the Viterbi algorithm. While iterating through the password string, the algorithm will navigate as far down the trie as possible with a next-node lookahead to try to find the word with the lowest cost. This will continue for each word discovered until a couple possible tokenizations of the password are discovered. From these possible tokenizations, the one with the lowest combined cost will be selected as the correct one. With this algorithm, the example password **LemonJumpSlide** could yield the possible tokenizations **[Lemon, JumpS, lide]** as well as **[Lemon, Jump, Slide]**. Since **lide** is a far more uncommon word than **slide**, the first tokenization will have a much higher total cost than the second tokenization, thus the second will be selected as the correct one.

Once tokenized, word tokens are the words discovered by the hybrid search, number tokens are every sequence of uninterrupted numerical digits, and special character tokens are simply what is left over. From this algorithm, the password **Password123!** will get split into tokens **Password**, **123**, **!**, a word token, number token, and special character token respectively.

Algorithm 3 Password Tokenization

procedure TOKENIZEPASSWORD(P)
  Input: Password string P
  Output: Sequence of tokens T
  $i \leftarrow 0$
  $T \leftarrow \emptyset$
  while $i < |P|$ **do**
    if isWord($P[i]$) **then**
      $c_{min} \leftarrow \infty$
      $V \leftarrow$ GetUnleetedVariants($P[i \dots |P|]$)
      for each $v \in V$ **do**
        $W \leftarrow$ FindPossibleWords(v)
        for each $w \in W$ **do**
          $c_{next} \leftarrow$ GetLowestCost($P[i+|w| \dots]$)
          $c_{total} \leftarrow$ Cost(w) $+ c_{next}$
          if $c_{total} < c_{min}$ **then**
            $c_{min} \leftarrow c_{total}$
            $token \leftarrow w$
          end if
        end for
      end for
    else if isNumber($P[i]$) **then**
      $token \leftarrow$ ExtractConsecutiveNumbers($P[i \dots]$)
    else
      $token \leftarrow P[i]$ ▷ Special character case
    end if
    $T \leftarrow T \cup \{token\}$
    $i \leftarrow i + |token|$
  end while
  return T
end procedure

B. Diction Perturbation

Each individual token is perturbed according to its distributed diction budget B_{dict}. Token perturbation is defined depending on the type of token: a word, number, or special character. An output space $\mathcal{R}$ is defined depending on the token type, and a utility score is assigned to each output depending on a scoring function $u(D, r)$. The probability that the token is perturbed to an output in an output space is defined as proportional to the exponential mechanism and an output is selected according to these probabilities.

For the word token type, the output space is defined as the 10 nearest words to the original word in the trie, defined by edge distance. The score function for this token type is the negative edge distance from the potential output to the original word. This score function algorithm is detailed in Algorithm 4.

Algorithm 4 Utility Score Assignment for Word Perturbation

procedure ASSIGNREWARD($\mathcal{W}$)
  Input: A set of candidate words $\mathcal{W} = \{w_0, w_1, \ldots, w_k\}$, where w_0 is the original word
  Output: The set $\mathcal{W}$ with updated reward values r
  $c_{min} \leftarrow \infty$
  for each $w \in \mathcal{W}$ **do**
    if cost$(w) < c_{min}$ **then**
      $c_{min} \leftarrow$ cost(w)
    end if
  end for
  cost$(w_0) \leftarrow c_{min}$ ▷ Normalize original word cost
  for each $w \in \mathcal{W}$ **do**
    $w.r \leftarrow c_{min} -$ cost$(w) -$ dist(w, w_0)
  end for
  return $\mathcal{W}$
end procedure

For number tokens, the length of the number represented in decimal is compared to all other possible numbers of the same length. For example, the number '67' has a length of two and is compared to all numbers of length two (10-99). The utility score in the case of numbers is the negative absolute value of the distance from the original number. For special character tokens, the layout of all special characters on a standard QWERTY computer keyboard is evaluated. The utility score is the negative absolute value of the distance from the original character on a keyboard.

The space $\mathcal{R}$ of possible perturbed outputs, their utility scores, and $\varepsilon = B_{dict}$ are fed into the exponential mechanism to determine the token chosen for perturbation.

C. Semantic Perturbation

To perform semantic perturbation, the Markov chain Monte Carlo (MCMC) Metropolis-Hastings [33] Algorithm is first used to generate a sample distribution of all possible permutations of tokens. Using the MCMC Metropolis-Hastings Algorithm is necessary to prevent a $O(n!)$ space complexity, which would be present if a brute-force algorithm were used to determine all possible permutations of tokens.

Each perturbed token is shuffled within the password string according to the privacy budget $\varepsilon = B_{sem}$. The utility score for this perturbation is defined as the negative total swapping distance (i.e. the number of adjacent token swaps to get from the original token list to the permuted token list).

D. Algorithm Results

Understanding how the parameters, namely ε, impact the results of this algorithm are critical for proper usage within a red team environment. Prior literature has accomplished many mechanisms for determining an optimal value of ε, [34] [35], however with a novel solution for achieving ε-LDP, such as LDPasswd, understanding the output of the algorithm is an important prerequisite for making an informed decision about ε.

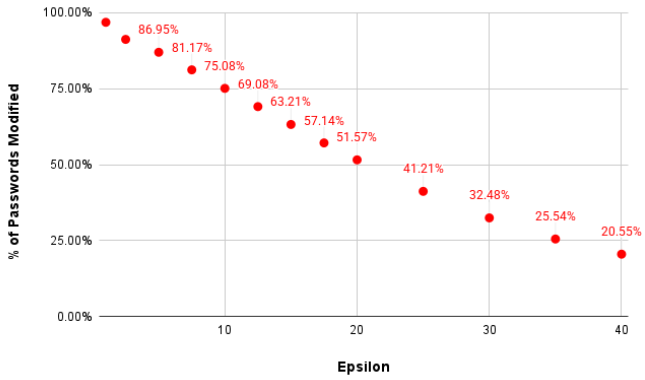


Fig. 1. The Effect of the Privacy Budget ε on the % of Passwords Perturbed

Figure 1 shows how often the resulting perturbed password from Algorithm 2 is different than the original password passed in as a parameter. Utilizing a probabilistic based method has been shown to be an effective method for explaining an optimal ε to non-privacy aware users [36], such as red team operators.

A wide range of ε values can be utilized depending on the privacy needs of the engagement environment. At the extremes, if privacy is of the utmost importance to the red team operators, utilizing $\varepsilon = 1$ will result in 96.81% of passwords being different than previously. As an example, using LDPasswd to perturb passwords **Password123!** and **LemonJumpSlide1#** ten times results in the following respective perturbed passwords for each:

```
}111Passer3!     }9LemaySlideJuba
Password]131     Lemon`1SliceJuan
~122Passer3!     %5JumpLemonSligh
134Passes$3!     ~8SliceJuanLemec
Passey-1213!     Jump3Lemon(Slide
}Password122     4JumoSlimsLemay!
Passes$1313!     JumpLemin5Slime`
!110Passi23!     #SlimyJumbo7Lemec
%Password116     SlifeJubLemar@9#
Password^108     Slide8LeminJuan&
```

As can be seen above, the use of $\varepsilon = 1$ removes nearly all utility from the password being collected, except to prove the fact that the red team was able to collect the passwords, which may be the objective of certain red team environments. On the opposite side, utilizing $\varepsilon = 40$ will result in only 20.55% of passwords being changed by LDPasswd, yielding the following passwords when using **Password123!** and **LemonJumpSlide1#** again as examples:

```
Password123!    LemonJump1Slide#
Password123!    LemonJumpSlide1#
Password123~    LemonJumpSlide1#
Password123!    LemonJumpSlide1#
Password123!    LemonJumpSlide1#
Password123!    LemonJumpSlide1#
Password123!    LemonJumpSlide1#
Password123!    LemonJumpSlide1@
123Password!    LemonJumpSlide1#
Password123!    LemonJumpSlide1#
```

It is clear that $\varepsilon = 40$ provides almost no privacy benefits except to some very simple membership inference attacks performed by attackers with practical knowledge (non-worst case scenario) [37], however this threat is not in the scope of attacks that LDPasswd is attempting to defend against. More realistic values of ε might be 10 or 30. When $\varepsilon = 30$, approximately 32.48% of passwords are changed by LDPasswd, and example passwords of **Password123!** and **LemonJumpSlide1#** result in the following potential perturbed passwords:

```
Password123!    LemonJump1Slide#
Password123!    LemonJumpSlide1#
123Password!    LemonJumpSlide1#
Password123!    LemonJumpSlide1#
Password123!    LemonJumpSlide1$
Password124!    LemonJumpSlide1#
Password123!    LemonJump1$Slide
Password123!    LemonJumpSlide1#
Password123!    LemonJumpSlide2#
Password123!    LemonJumpSlide1$
```

$\varepsilon = 30$ provides more privacy than $\varepsilon = 40$. $\varepsilon = 10$ and the same example passwords result in the following perturbed passwords.

```
Password123!    LemonJump4^Sligo
123Password@    LemonJumpSlide2#
@Password121    LemonJumpSlide#1
Password@123    Jump2SlideLemon#
Password123!    LemonJumpSlide1#
Passes$123      Lemon3Jump$Slide
122Password~    LemonJumpSlide4~
!Password123    LemonJump1Slide#
123!Password    LemonJumpSlide1#
Password123~    JumpLemonSlide`#
```

$\varepsilon = 10$ maintains more utility than $\varepsilon = 1$, however it is unclear whether or not enough utility is preserved to ensure that red team operators are not affected. In order to determine how much utility is truly preserved by various values of ε, a statistical experiment is necessary.

VI. Experimentation

A. Motivation

It is established that LDPasswd fulfills ε-local differential privacy. However, without a formal experiment, it is unclear how much Red Team operator utility is affected impacted due to the perturbation of credentials. At first, it is not obvious why the utility lost is not the same as the proportion of passwords that are incorrect. In reality, the utility lost is much lower than that, and there are multiple reasons for this, some of the biggest being:

1) Red Team often patches existing authentication mechanisms in such a manner that any password is considered correct, thus bypassing authentication entirely.
2) Blue teams (defenders) rapidly change their passwords after discovering and removing the aforementioned malicious software. This means that many of the passwords collected by the Red Team are already incorrect.
3) Red Team often does not use passwords at all to access compromised machines - malware that require no authentication at all.

To evaluate how much utility is lost while using LDPasswd, the RITSEC Red Team [38] conducted an experiment at Lockdown [39], a red vs. blue cybersecurity competition hosted by UBNetDef [40](The University at Buffalo's network security club) Lockdown 2026 was hosted on April 11, 2026, with 14 different top collegiate programs [39] competing as blue teams. These competing blue teams [39] included multiple time Collegiate Cyber Defense Competition (CCDC) National Champions and other National CCDC finalists [41].

B. Hypotheses

The following hypotheses were proposed regarding the effects of perturbation on authentication success:

1) The authentication success rate is not independent, but is dependent on the value of the privacy budget ε.
2) There is not a statistically significant difference between the control group and ε = 30, but there is between the other groups.
3) The Red Team will still be just as effective in its end goal of maintaining persistent access to all machines.

C. Experiment Methodology

Lockdown's competition environment consisted of 8 different systems per team: one router, 4 Linux servers, and 3 Windows servers. To achieve the most accurate results possible, each team was targeted similarly during competition and experimental groups were treated equally across Windows and Linux.

To conduct the experiment, RITSEC Red Team malicious password filters ZeroPAM [42] and WinFilter [43] were deployed to harvest credentials on Linux and Windows respectively. For the experiment, the credential harvesting tools were deployed on 98 total machines, configured to perform perturbation of harvested passwords locally using LDPasswd. The machines were distributed evenly across three groups:

- a control group (no perturbation)
  - three machines per team
  - two Linux, one Windows
- a group with $\varepsilon = 30$
  - two machines per team
  - one Linux, one Windows
- a group with $\varepsilon = 10$
  - two machines per team
  - one Linux, one Windows

Each time an operator attempted to use a harvested password, the result of the authentication attempt (success/failure) was recorded.

D. Analysis

Group	Success	Failure	Total
Control	12	1	13
ε=30	5	1	6
ε=10	1	5	6
Total	18	7	25

Fig. 2. Categorized Results from Experiment

Figure 2 contains the categorized results from the experiment. To determine if the proportion of authentication successes is dependent upon the group, a Chi-Square Test for Independence was conducted [44]. This type of statistical test compares the expected counts of categorical data against the actual counts of the data to determine whether or not one variable is independent of another. In this case, it determines if the authentication success rate is independent of the experimental group that the system is in. Appendices A-A and B-A show the relevant calculations - a p-value of less than α was achieved, causing rejection of the null hypothesis and thus statistically proving that the proportion of authentication successes depends on the experimental group.

Now that dependence has been established, three 2-sample Z-tests [45] are performed to check for statistically significant differences in the sample proportions of authentication successes between the three experimental groups. From Appendixes B-C, B-B, and B-D, the null hypothesis fails to be rejected when comparing $\varepsilon = 30$ and the control group (meaning there is not a statistically significant difference between these experimental groups) and is rejected the other two cases (meaning there is statistically significant difference between $\varepsilon = 10$ and the other two experimental groups). In other words, no utility is lost to the red team when $\varepsilon = 30$, but there is a non-zero amount of utility that is lost when $\varepsilon = 10$.

Both hypothesis one and two have been proved correct. Finally, the red team was able to accomplish a successful "red board" at Lockdown (taking down all blue team services at the end of competition), proving hypothesis three correct, and showing even with a mathematical privacy protection of passwords, the red team's ability to be an effective adversary was not affected.

E. Limitations

There are two major limitations of the study. The first limitation is the limited sample size for our tests. For each experimental group / categorical variable combination in a chi-squared independence test, it is recommended to have a value of at least 5 for each observed count. As seen in A-A, only 3/6 combinations have a value of at least 5 for their observed counts. Additionally, the sample size recommendations for Z-tests are not met either. For a Z-test for two sample proportions, it is recommended that both $n(\hat{p}) \geq 10$ and $n(1 - \hat{p}) \geq 10$, but as seen in Appendix B-E, only 1/12 of the recommended sample sizes is met. These sample size limitations, however, do not invalidate our Chi-Squared test and Z-test results. These recommendations are simply to obtain the most accurate p-value possible when comparing to α. Because all of the p-values seen in Appendices B-A, B-D, B-C, and B-B are all so drastic and far away from α, it can be assumed that this lack of meeting sample size recommendations has no effect on the study's inferences from the statistical tests.

Another potential limitation from our study and statistical analysis is lack of analysis of the feasibility of reconstruction attacks. Reconstruction attacks against Local Differential Privacy occur when an user, who only has access to the noisy data, is able to reconstruct the true data owned by the user. Such attacks have been shown to work against various implementations of DP, such as LDP in Federated Learning [46] and DP with relaxed attacker assumptions [47]. In regards to LDPasswd, the worry is that a potential reconstruction attack exists when red team operators choose to use a high value of epsilon. For example, when a red team operator sees the password **Password122!** and attempts to authenticate with it and authentication fails, a natural thought might be to try **Password123!** as a password instead. If authentication succeeds, the operator as just performed a reconstruction attack. While no formal experiment and statistical analysis occurred to prove whether or not such attacks were feasible, during the Lockdown competition multiple operators did attempt to perform reconstruction attacks, and none were successful. None of these authentication results were formally documented however, so no results can be formally drawn from this for the study.

VII. Conclusion

This work introduced LDPasswd, a novel library designed to guarantee ε-Local Differential Privacy for password collection within simulated adversarial cybersecurity engagements. By integrating tokenization through a mini Viterbi algorithm, diction perturbation of individual tokens via the exponential mechanism, and semantic perturbation through MCMC-based sampling, LDPasswd effectively obfuscates sensitive password semantics to preserve privacy of simulation participants.

The experimental evaluation demonstrates that privacy-preserving perturbation can be applied without significantly degrading red team effectiveness, provided that the privacy budget remains within a moderate range. Specifically, results

indicate that at $\varepsilon = 30$, there is no statistically significant impact on authentication success rates compared to the unperturbed control group. However, stronger privacy guarantees (e.g. $\varepsilon = 10$) introduce measurable degradation in utility.

Importantly, these findings validate that local differential privacy can be feasibly integrated into real-world red team tooling without undermining the core objectives of adversarial simulation. Furthermore, the study supports the broader claim that privacy-preserving mechanisms can mitigate unintended psychological and security risks to participants, particularly in environments where real or semantically meaningful passwords may be inadvertently exposed.

Overall, LDPasswd provides a practical and mathematically grounded approach to safeguarding user privacy in cybersecurity exercises. This work contributes to bridging the gap between theoretical privacy guarantees and applied offensive security practices, demonstrating that strong privacy protections and realistic adversarial utility are not mutually exclusive but can be jointly achieved through careful system design.

VIII. Future Work

Although this research demonstrated the efficacy of implementing Local Differential Privacy (LDP) for password collection, there is still room to expand in the future.

The tools used for password collection in this study were a custom DLL that was injected into the LSASS process (WinFilter) and a Linux-based PAM shim (ZeroPAM). However, a Linux passwd shim tool was not utilized due to testing constraints, and neither of these tools currently work on macOS or FreeBSD. Therefore, expanding the toolset to include a Linux passwd shim and implementing macOS and FreeBSD compatibility represents a critical next step. Successful porting would further prove that the LDP framework is platform-agnostic, meaning that the privacy protections remain effective regardless of the underlying operating system's architecture.

Beyond technical development, future research should also include improvements on the statistical inference portion of this study. First, conducting A longitudinal study with a significantly larger sample size would further validate the statistical utility of this LDP implementation. While the current study demonstrates the effectiveness of the approach within a controlled environment, increasing the volume of participants would allow for a more granular analysis of how different epsilon (ε) privacy budgets impact the accuracy of password distribution modeling at scale. Additionally, further studies should test more values of ε. This work was limited due to the size of the Lockdown infrastructure, but conducting further work across both multiple simulated adversary environments, as well as larger simulated adversary environments would solve this issue. Finally, statistical analysis should be conducted on the feasibility of reconstruction attacks, to determine potential areas of risk compared to when using high values of (ε).

Acknowledgments

This work was supported by University at Buffalo's annual Lockdown competition. Due to the event organizer's generosity for allowing this work to be conducted, LDPasswd is able to demonstrate real-world applications in data privacy and cybersecurity engagements.

References

[1] N. Veerasamy, "High-level methodology for carrying out combined red and blue teams," in *2009 Second International Conference on Computer and Electrical Engineering*, vol. 1, 2009, pp. 416–420.

[2] C. Chindrus and C.-F. Caruntu, "Securing the network: A red and blue cybersecurity competition case study," *Information*, vol. 14, no. 11, 2023. [Online]. Available: https://www.mdpi.com/2078-2489/14/11/587

[3] C. CHINDRUȘ and C.-F. CĂRUNTU, "Enhancing cybersecurity readiness through the red and blue team competition," 2023.

[4] V. Baladari, "Red team vs. blue team: Assessing cybersecurity resilience through simulated attacks," *European Journal of Advances in Engineering and Technology*, vol. 8, no. 4, pp. 82–87, 2021.

[5] S. Yulianto, B. Soewito, F. L. Gaol, and A. Kurniawan, "Enhancing cybersecurity resilience through advanced red-teaming exercises and MITRE ATT&CK framework integration: A paradigm shift in cybersecurity assessment," *Cyber Security and Applications*, vol. 3, p. 100077, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2772918424000432

[6] D. Berardi, F. Callegati, A. Melis, and M. Prandini, "Password similarity using probabilistic data structures," *Journal of Cybersecurity and Privacy*, vol. 1, no. 1, pp. 78–92, 2020.

[7] M. Learn, "Enhancing cybersecurity resilience through advanced red-teaming exercises and mitre att&ck framework integration: A paradigm shift in cybersecurity assessment," *Microsoft Learn*, 2026. [Online]. Available: https://learn.microsoft.com/en-us/windows-server/security/credentials-protection-and-management/configuring-additional-lsa-protection

[8] K. Geisshirt, "Pluggable authentication modules," *Birmingham, UK: Packt Publishing*, vol. 105, 2007.

[9] MITRE, "Modify Authentication Process: Password Filter DLL," *MITRE ATT&CK Framework*, 2026. [Online]. Available: https://attack.mitre.org/techniques/T1556/002

[10] ——, "Modify Authentication Process: Pluggable Authentication Modules," *MITRE ATT&CK Framework*, 2026. [Online]. Available: https://attack.mitre.org/techniques/T1556/003

[11] ——, "Mirrorface," *MITRE ATT&CK Groups*, 2026. [Online]. Available: https://attack.mitre.org/groups/G1054/

[12] ——, "Oilrig," *MITRE ATT&CK Groups*, 2026. [Online]. Available: https://attack.mitre.org/groups/G0049/

[13] ——, "Ebury," *MITRE ATT&CK Software*, 2026. [Online]. Available: https://attack.mitre.org/software/S0377/

[14] ——, "Skidmap," *MITRE ATT&CK Software*, 2026. [Online]. Available: https://attack.mitre.org/software/S0468/

[15] C. Dwork, "The differential privacy frontier (extended abstract)," in *Theory of Cryptography. Sixth Theory of Cryptography Conference, TCC 2009, San Francisco, CA, USA, March 15-17, 2009, Proceedings*, 2009.

[16] L. Sweeney, "Achieving k-anonymity privacy protection using generalization and suppression," *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 05, pp. 571–588, 2002.

[17] E. Lantz, K. Boyd, and D. Page, "Subsampled exponential mechanism: Differential privacy in large output spaces," in *Proceedings of the 8th ACM Workshop on Artificial Intelligence and Security*, 2015, pp. 25–33.

[18] O. Klymenko, S. Meisenbacher, and F. Matthes, "Differential privacy in natural language processing: The story so far," in *Proceedings of the Fourth Workshop on Privacy in Natural Language Processing*, 2022, pp. 1–11.

[19] D. Jurafsky and J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*, 3rd ed., 2026, online manuscript released January 6, 2026. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/

[20] B. Ghojogh, F. Karray, and M. Crowley, "Hidden markov model: tutorial," 2019.

[21] Z. Hatala and V. Puturuhu, "Viterbi extraction tutorial with hidden markov toolkit," *arXiv preprint arXiv:1908.03143*, 2019.

[22] J. Blocki, A. Datta, and J. Bonneau, "Differentially private password frequency lists," *Cryptology ePrint Archive*, 2016.

[23] K. Panchal, "Differential privacy and natural language processing to generate contextually similar decoy messages in honey encryption scheme," *arXiv preprint arXiv:2010.15985*, 2020.

[24] Y. Li, H. Wang, and K. Sun, "Personal information in passwords and its security implications," *IEEE Transactions on Information Forensics and Security*, vol. 12, no. 10, pp. 2320–2333, 2017.

[25] M. Swanberg, M. S. M. S. Annamalai, J. Hayes, B. Balle, and A. Smith, "Beyond the worst case: Extending differential privacy guarantees to realistic adversaries," *arXiv preprint arXiv:2507.08158*, 2025.

[26] M. Naor, B. Pinkas, and E. Ronen, "How to (not) share a password: Privacy preserving protocols for finding heavy hitters with adversarial behavior," in *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, 2019, pp. 1369–1386.

[27] D. Borges, "Red teaming at National CCDC 2023," *LockBoxx: A Hacker's Blog*, 2023. [Online]. Available: https://lockboxx.blogspot.com/2023/05/red-teaming-at-national-ccdc-2023.html?ref=sshell.co

[28] J. Southcott and M. Marino, "LDPasswd," https://github.com/jamessouthycoder19/ldpasswd, 2026.

[29] Dropbox, "zxcvbn," https://github.com/dropbox/zxcvbn, 2017.

[30] J.-I. Aoe, K. Morimoto, and T. Sato, "An efficient implementation of trie structures," *Software: Practice and Experience*, vol. 22, no. 9, pp. 695–721, 1992.

[31] RITSEC, "IRSeC," 2026. [Online]. Available: https://irsec.club

[32] W. Li and J. Zeng, "Leet usage and its effect on password security," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 2130–2143, 2021.

[33] S. Chib and E. Greenberg, "Understanding the metropolis-hastings algorithm," *The american statistician*, vol. 49, no. 4, pp. 327–335, 1995.

[34] J. Hsu, M. Gaboardi, A. Haeberlen, S. Khanna, A. Narayan, B. C. Pierce, and A. Roth, "Differential privacy: An economic method for choosing epsilon," in *2014 IEEE 27th Computer Security Foundations Symposium*. IEEE, 2014, pp. 398–410.

[35] N. Kohli and P. Laskowski, "Epsilon voting: Mechanism design for parameter selection in differential privacy," in *2018 IEEE Symposium on Privacy-Aware Computing (PAC)*, 2018, pp. 19–30.

[36] P. Nanayakkara, M. A. Smart, R. Cummings, G. Kaptchuk, and E. M. Redmiles, "What are the chances? explaining the epsilon parameter in differential privacy," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 1613–1630.

[37] A. Lowy, Z. Li, J. Liu, T. Koike-Akino, K. Parsons, and Y. Wang, "Why does differential privacy with large epsilon defend against practical membership inference attacks?" *arXiv preprint arXiv:2402.09540*, 2024.

[38] R. R. Team, "RITSEC red team wiki," *RITSEC*, 2026. [Online]. Available: https://ritredteam.github.io/

[39] UBNetDef, "Lockdown," *UBNetDef*, 2026. [Online]. Available: https://lockdown.ubnetdef.org/

[40] ——, "University at buffalo network defense is a cyber security class club," *UBNetDef*, 2026. [Online]. Available: https://ubnetdef.org/

[41] NCCDC, "NCCDC - Past Winners," 2026. [Online]. Available: https://www.nationalccdc.org/winners.html

[42] M. Marino, "ZeroPAM," https://github.com/max-49/zeropam, 2026.

[43] Enzo DeStephano, "WinFilter," 2026. [Online]. Available: https://github.com/kindtime/winfilter

[44] M. F. Zibran, "Chi-squared test of independence," *Department of Computer Science, University of Calgary, Alberta, Canada*, vol. 1, no. 1, pp. 1–7, 2007.

[45] D. N. Lawley, "A generalization of fisher's z test," *Biometrika*, vol. 30, no. 1/2, pp. 180–187, 1938.

[46] Z. You, X. Dong, S. Li, X. Liu, S. Ma, and Y. Shen, "Local differential privacy is not enough: A sample reconstruction attack against federated learning with local differential privacy," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 1519–1534, 2024.

[47] P. Protivash, J. Durrell, Z. Ding, D. Zhang, and D. Kifer, "Reconstruction attacks on aggressive relaxations of differential privacy," *arXiv preprint arXiv:2209.03905*, 2022.

APPENDIX A
TABLES

A. Expected Counts

Group	Success	Failure
Control	$\frac{13*18}{25} = 9.36$	$\frac{13*7}{25} = 3.64$
ε=30	$\frac{6*18}{25} = 4.32$	$\frac{6*7}{25} = 1.68$
ε=10	$\frac{6*18}{25} = 4.32$	$\frac{6*7}{25} = 1.68$

APPENDIX B
STATISTICAL INFERENCE

A. Chi-Squared Independence Test

$$x_0^2 = \sum_{i=1}^{a}\sum_{j=1}^{b} \frac{(obs_{ij} - exp_{ij})^2}{exp_{ij}}$$

$$x_0^2 = \frac{(12-9.36)^2}{9.36} + \frac{(1-3.64)^2}{3.64} + \frac{(5-4.32)^2}{4.32} + \frac{(1-1.68)^2}{1.68} + \frac{(1-4.32)^2}{4.32} + \frac{(5-1.68)^2}{1.68} = 12.154$$

$$df = (3-1)(2-1) = 2$$

$P(X_2^2 > 12.154) = .003$ which is less than .05

Reject H_0

B. Testing Difference Between ε=30 (2) and the Control Group (1)

$$x_2 = 5, n_2 = 6, x_1 = 12, n_1 = 13$$

$$\hat{p}_2 = 5/6 = .83, \hat{p}_1 = 12/13 = .923$$

$$\hat{p} = \frac{x_1 + x_2}{n_1 + n_2} = \frac{5+12}{6+13} = \frac{17}{19} = .895$$

$$\Delta_0 = 0, \alpha = .05$$

$$z_o = \frac{\hat{p}_2 - \hat{p}_1 - \Delta_0}{\sqrt{\hat{p}(1-\hat{p})(\frac{1}{n_2} + \frac{1}{n_1})}} = \frac{.83 - .923 - 0}{\sqrt{.895(1-.895)(\frac{1}{13} + \frac{1}{6})}} = -.593$$

$$P(z_o < -.593) = .27680$$

$.27680 > .05$

Fail to Reject H_0

C. Testing Difference Between ε=10 (2) and the Control Group (1)

$$x_2 = 1, n_2 = 6, x_1 = 12, n_1 = 13$$

$$\hat{p}_2 = 1/6 = .167, \hat{p}_1 = 12/13 = .923$$

$$\hat{p} = \frac{x_1 + x_2}{n_1 + n_2} = \frac{1 + 12}{6 + 13} = \frac{13}{19} = .684$$

$$\Delta_0 = 0, \alpha = .05$$

$$z_o = \frac{\hat{p}_2 - \hat{p}_1 - \Delta_0}{\sqrt{\hat{p}(1-\hat{p})(\frac{1}{n_2} + \frac{1}{n_1})}} = \frac{.167 - .923 - 0}{\sqrt{.684(1-.684)(\frac{1}{6} + \frac{1}{13})}} = -3.297$$

$P(z_o < -3.297) = .00049$ which is less than .05

Reject H_0

D. Testing Difference Between ε=10 (2) and ε=30 (1)

$$x_2 = 1, n_2 = 6, x_1 = 5, n_1 = 6$$

$$\hat{p}_2 = 1/6 = .167, \hat{p}_1 = 5/6 = .83$$

$$\hat{p} = \frac{x_1 + x_2}{n_1 + n_2} = \frac{1 + 5}{6 + 6} = \frac{6}{12} = .5$$

$$\Delta_0 = 0, \alpha = .05$$

$$z_o = \frac{\hat{p}_2 - \hat{p}_1 - \Delta_0}{\sqrt{\hat{p}(1-\hat{p})(\frac{1}{n_2} + \frac{1}{n_1})}} = \frac{.167 - .83 - 0}{\sqrt{.5(1-.5)(\frac{1}{6} + \frac{1}{6})}} = -2.309$$

$P(z_o < -2.309) = .0105$ which is less than .05

Reject H_0

E. Z-Test Sample Size Limitations

ε=30 and Control Group

$$\hat{p} = .895, n_2 = 6, n_1 = 13$$

$$n_2(\hat{p}) = 6(.895) = 5.37$$

$$n_2(1-\hat{p}) = 6(.105) = .63$$

$$n_1(\hat{p}) = 13(.895) = 11.622$$

$$n_1(1-\hat{p}) = 13(.105) = 1.365$$

ε=10 and Control Group

$$\hat{p} = .684, n_2 = 6, n_1 = 13$$

$$n_2(\hat{p}) = 6(.684) = 4.104$$

$$n_2(1-\hat{p}) = 6(.316) = 1.896$$

$$n_1(\hat{p}) = 13(.684) = 8.892$$

$$n_1(1-\hat{p}) = 13(.316) = 4.108$$

ε=10 and ε=30

$$\hat{p} = .5, n_2 = 6, n_1 = 6$$

$$n_2(\hat{p}) = 6(.5) = 3$$

$$n_2(1-\hat{p}) = 6(.5) = 3$$

$$n_1(\hat{p}) = 6(.5) = 3$$

$$n_1(1-\hat{p}) = 6(.5) = 3$$